Brittleness Under Refactoring: A Dafny Benchmark and Empirical Study

[MISSING: Author names]

[1] **[MISSING: Affiliation 1]**, **[MISSING: City]**, **[MISSING: Country]**
[MISSING: Author 1 email]
[2] **[MISSING: Affiliation 2]**, **[MISSING: City]**, **[MISSING: Country]**
[MISSING: Author 2 email]

Abstract. Verified software evolves, but the maintenance cost of preserving verification under routine structural change remains poorly characterized. This paper introduces Dataset A, a maintenance-centred Dafny benchmark for nearby-version proof breakage under semantics-preserving refactoring. The benchmark organizes transformed instances by seed-program stratum, refactoring class, verification outcome, and failure-mode label, and it evaluates three simple recovery baselines—re-verification, syntactic transfer, and LLM-only repair—to measure how much of the resulting breakage can be removed without structured reasoning. The paper's contribution is therefore empirical rather than algorithmic: it turns proof brittleness under software evolution into a reproducible benchmark problem and uses that benchmark to expose the residual repair gap that remains after lightweight recovery.

Keywords: formal verification · proof maintenance · refactoring · Dafny · empirical software engineering

Artifact availability. Artifact URL: **[MISSING: Artifact URL]**. Anonymized package: **[MISSING: Anonymous artifact package name]**. Docker image: **[MISSING: Docker image reference]**. Replay command: **[MISSING: Replay command]**.

1 Introduction

Verified software does not remain static. As code evolves, specifications, assertions, invariants, and proof hints must evolve with it. In mainstream software engineering, refactoring is a routine part of maintenance rather than an exceptional event [14], and empirical studies show that real projects perform a wide range of such structural changes during ordinary development [10, 18]. More broadly, maintenance research has emphasized that software artifacts are best understood through co-evolution across versions rather than as isolated one-shot tasks [25]. For verified programs, however, one basic empirical question remains underexplored: how brittle is verification under semantics-preserving refactoring?

This question is especially salient in auto-active verification settings such as Dafny, where program text and proof-relevant structure are intertwined in a single workflow [11]. Dafny automates much of proof checking, but successful verification still depends on carefully arranged contracts, loop invariants, assertions, and ghost code [7]. Recent studies of Dafny practice suggest that maintainability and workflow friction remain central to the practical cost of verification adoption [16]. The issue, then, is not proving from scratch, but understanding how nearby-version edits disrupt proofs that previously discharged automatically.

Existing literature only partially addresses this problem. Refactoring is well studied in software engineering, and proof refactoring has been explored in interactive theorem proving [20, 21, 1]. The Dafny ecosystem has also seen recent work on benchmarks, testing, and learning-based support [13, 19, 23]. What remains missing is a reproducible benchmark and empirical characterization of *semantics-preserving refactoring-induced proof breakage in already verified Dafny programs*. This paper targets that gap.

To do so, it introduces Dataset A, a maintenance-centred benchmark of verified Dafny programs paired with transformed variants drawn from seven refactoring classes. The benchmark is used to measure how often verification breaks, how failures cluster by proof-relevant failure mode, and how much breakage simple baseline recovery can repair. The baselines are informed by automated repair and proof-automation work [6, 22, 4, 17, 15], but the paper is not itself a repair-method contribution. Its central claim is narrower and, we argue, foundational: nearby-version proof breakage is a measurable maintenance problem, and lightweight recovery strategies leave a substantial residual repair gap.

Our contributions are:

- a reproducible Dafny benchmark for refactoring-induced proof brittleness in already verified programs;
- an empirical characterization of breakage rates and failure-mode clusters across seven semantics-preserving refactoring classes;
- a baseline comparison of re-verification, syntactic transfer, and LLM-only repair that quantifies the residual repair gap while also providing an evaluation substrate for later repair work.

The rest of the paper develops this argument in a compact sequence. Section 2 clarifies the conceptual framing needed to separate nearby-version proof brittleness from other forms of change, Section 3 describes Dataset A and the refactoring benchmark, Section 4 specifies the baselines and evaluation setup, and Section 5 presents the empirical results and their intended interpretation. Section 6 discusses threats to validity, Section 7 positions the paper against related work, and Section 8 concludes by summarizing why the benchmark is a standalone contribution as well as a foundation for later repair work.

2 Background and Study Framing

2.1 Proof Maintenance Under Evolution

In this paper, *proof maintenance* means preserving or restoring successful verification across nearby program versions rather than proving each version from scratch. That framing matters because verified software evolves alongside the rest of a codebase: implementations change, helper abstractions move, and proof-relevant artifacts must co-evolve with the source program [25]. We distinguish three notions throughout the paper. *Source evolution* refers to the program change itself. *Proof brittleness* refers to the failure of previously successful verification under a nearby-version edit, even when the intended behavior remains unchanged. *Semantic drift* refers instead to cases where the meaning of the program or its intended specification has genuinely changed. Keeping those notions separate matters empirically: without that distinction, a benchmark could blur maintenance failures together with genuine redesign. Prior work on proof refactoring shows that maintenance of proof artifacts is a legitimate research problem in its own right [20, 21]; this paper focuses on the empirical characterization of that problem in an auto-active verification setting rather than on automated repair.

2.2 Why Dafny

Dafny is an appropriate setting for this study because it makes proof-relevant structure visible at the source level while still relying on largely automatic proof checking [11]. Contracts, assertions, loop invariants, and ghost code appear directly in the program text, so maintenance effects are observable without requiring access to low-level interactive proof objects. At the same time, successful Dafny verification still depends on structured proof guidance and recurring verification idioms [7]. Recent empirical work also suggests that usability and maintenance effort remain central to the practical cost of formal verification adoption in Dafny [16]. This makes Dafny especially suitable for a benchmark whose purpose is not merely to count failures, but to examine which kinds of source-visible proof structure are most brittle under evolution.

2.3 Semantics-Preserving Refactoring in This Paper

The phrase *semantics-preserving refactoring* is used narrowly in this paper. Following the general refactoring literature [14, 10, 18], we study transformations intended to preserve program behavior while changing source structure. In this paper, that definition is further restricted to refactorings that may perturb verification outcomes by moving, reshaping, or redistributing proof-relevant structure in already verified Dafny programs. This excludes verifier bugs, deliberate semantic changes, intentionally weakened specifications, and unconstrained real-history edits. The point of controlled semantics-preserving refactoring here is not to model all software evolution, but to create nearby-version pairs with

clean enough ground truth that proof brittleness can be measured reproducibly and compared across baselines.

2.4 Study Objective and Evaluation Lens

This paper studies how often verification breaks under controlled semantics-preserving refactoring, how those failures cluster by proof-relevant failure mode, and how much simple baseline recovery can restore. Dataset A is therefore treated as a maintenance-centred benchmark rather than a plain corpus of transformed programs: each item is a nearby-version pair annotated with refactoring class, verification outcome, and baseline-recovery information. The evaluation lens is correspondingly simple and explicit: benchmark composition, breakage rate, baseline recovery, and the resulting residual repair gap. Repair and proof-automation work provide useful comparison points for baseline design [6, 4, 17], but the paper does not attempt to solve the repair problem itself. Instead, it defines the empirical problem sharply enough that Section 3 can introduce Dataset A as a reusable substrate for studying nearby-version proof breakage under software evolution.

3 Dataset and Refactoring Benchmark

3.1 Dataset A Overview

Dataset A is the core artifact of this paper: a maintenance-centred benchmark for nearby-version proof breakage in verified Dafny programs. Each benchmark item is organized around a verified seed program, a semantics-preserving refactored variant, the resulting verification outcome, and the outcomes of the baseline-recovery procedures introduced in Section 4. This definition makes the benchmark unit explicitly revision-aware. Rather than treating verification as a one-shot proving task, Dataset A captures what happens when a previously verified program is subjected to controlled source evolution and then replayed under the same verification workflow. In that respect, it complements recent Dafny benchmark efforts aimed at benchmark generation, synthesis, or compositional reasoning rather than maintenance under evolution [13, 23, 19]. The benchmark's value is therefore not only that it contains transformed programs, but that it preserves enough structure to support causal interpretation of how and why verification breaks.

3.2 Seed Corpus and Strata

The seed corpus is restricted to already verified Dafny programs whose proof-relevant structure is visible at the source level through contracts, assertions, loop invariants, and ghost code [11, 7]. The goal is not to maximize raw syntax diversity, but to assemble examples that expose plausible maintenance stress when their structure changes. To support later stratified analysis, Dataset A groups

seeds into three operational strata: tutorial and educational examples, benchmark or example-suite programs, and proof-engineering-heavy programs with substantial manual annotation. This split distinguishes lightweight pedagogical cases from instances that exercise more realistic proof maintenance behavior. It also keeps the benchmark from collapsing into a single style of Dafny development and makes it possible to ask whether nearby-version proof breakage concentrates in one source regime or persists across multiple levels of proof engineering maturity. Where available, ecosystem tooling and testing work provide additional evidence that Dafny now has a broad enough surrounding infrastructure to support this style of empirical study [9, 2].

3.3 Benchmark Design

The main design choice behind Dataset A is to use controlled refactorings instead of unconstrained repository history. This choice trades away some messiness of real-world evolution in exchange for reproducibility, clearer causal interpretation, and benchmark items with cleaner ground truth. In the software-engineering literature, refactoring denotes structural change intended to preserve behavior [14, 10, 18]. This paper adopts that idea in a narrower form suited to verification maintenance: each benchmark item is built to isolate whether source-level structural change destabilizes an existing verified program even when the intended behavior is unchanged.

This makes Dataset A scientifically different from generation-oriented Dafny benchmarks [13, 23, 19]. Those benchmarks focus on constructing or solving verification tasks from scratch; Dataset A instead measures maintenance under evolution. Its explicit axes are the seed-program stratum, the refactoring class applied to the seed, the post-refactoring verification outcome, and the baseline-recovery outcome recorded after replay. The benchmark therefore excludes deliberate semantic changes, verifier bugs as the primary object of study, and unconstrained edit histories whose causal structure is too noisy for the paper's central question.

3.4 Refactoring Catalog

P1 studies seven semantics-preserving refactoring classes chosen to span a range of plausible maintenance stresses rather than to exhaust all possible evolution events. The first group consists of local structural rewrites that mostly preserve control flow while perturbing proof-relevant names or local context: renaming changes the symbolic surface seen by transferred hints, helper extraction moves intermediate reasoning into a new abstraction boundary, and inlining collapses an existing abstraction back into its caller. A second group reshapes the local control or data context in which proofs discharge: legal statement reordering alters the order in which facts become available, non-behavioral loop reshaping perturbs loop-local reasoning without changing intended behavior, and contract factoring redistributes specification material across helper boundaries or shared clauses. The final group targets proof-context displacement directly: ghost-code

movement relocates proof hints, witness structure, or auxiliary lemmas while leaving executable behavior unchanged.

These classes are retained because each plausibly perturbs a different proof-relevant mechanism while remaining in scope as semantics-preserving refactoring. Renaming and ghost-code movement primarily test surface continuity of proof hints. Helper extraction, inlining, and contract factoring stress how verification obligations depend on local abstraction boundaries. Legal statement reordering and loop reshaping stress sensitivity to source order, invariant placement, and nearby solver context. Collectively, the catalog is intended to cover a meaningful spread of maintenance pressures on verified Dafny programs without conflating them with semantic edits or underspecified programs. That breadth matters for interpretation in Section 5: if brittleness appears across these distinct categories, the result is harder to dismiss as a quirk of a single transformation style.

3.5 Generation Pipeline

Each benchmark item is produced by a replayable four-stage pipeline. First, a candidate seed is validated to ensure that the original Dafny program verifies successfully under the recorded toolchain. Second, an eligible controlled transformation is applied to create a semantics-preserving nearby-version variant. Third, the transformed program is re-verified and its outcome is captured, including success, failure, timeout, or exclusion when the transformation does not apply cleanly. Fourth, the resulting pair is packaged with benchmark metadata so that all baselines operate over the same original program, transformed program, and verification context.

This pipeline makes the artifact boundary explicit. For every retained item, Dataset A stores the original program, the transformed program, the observed verification result after refactoring, a slot for a proof-oriented failure label, and slots for the baseline-recovery outcomes reported later in the paper. Cases that cannot be transformed into a valid nearby-version pair are logged and removed rather than silently repaired by hand. That policy keeps the benchmark interpretable as an evaluation substrate for maintenance under evolution and preserves comparability across baseline runs.

3.6 Failure-Mode Labels and Benchmark Outputs

Dataset A is intended to support more than a single breakage percentage, so broken instances are annotated with a compact set of failure-mode labels. The label set used in P1 distinguishes broken assertion flow, weakened loop invariants, brittle lemma or helper-application structure, trigger or quantifier sensitivity, ghost-code or proof-hint displacement, and unexplained or tool-sensitive failure. These labels are analytic categories for empirical characterization rather than definitive mechanistic diagnoses. Their purpose is to support stratified reporting, to make the residual repair gap interpretable, and to indicate which broken

cases are likely to require stronger forms of repair than simple replay or surface transfer.

At the artifact level, the benchmark outputs comprise the transformed instances themselves, their verification outcomes, their failure-mode labels when verification breaks, and the baseline-recovery outcomes and cost hooks used in Section 5. This makes Dataset A reusable as both a dataset and an evaluation substrate: a future repair system can be evaluated on the same nearby-version pairs, while this paper focuses on establishing the benchmark composition and the residual gap left by simple baselines. Section 4 builds directly on this structure by defining how baseline recovery is measured over each benchmark item.

4 Baselines and Experimental Setup

4.1 Why These Baselines

P1 evaluates three simple recovery strategies because the goal of the paper is to measure the residual repair gap, not to survey the full repair design space. Each baseline isolates a different notion of reuse over the nearby-version pairs in Dataset A. B1 captures the status quo in which a refactored program is simply re-verified without any intervention. B2 captures nearby-version continuity by attempting to reuse proof-relevant source artifacts from the seed version through surface-level transfer. B3 captures the strongest lightweight generation alternative by asking a modern language model to produce a repair directly from the broken transformed program and verifier feedback. Taken together, these baselines are sufficient to reveal whether semantics-preserving refactoring leaves a substantial amount of nearby-version proof breakage that later structured repair methods would still need to address.

4.2 Baseline Definitions

B1: Re-verification. B1 is the lower-bound baseline. For each Dataset A item, Dafny is rerun on the transformed program exactly as produced by the benchmark pipeline, with no attempt to edit, transfer, or repair proof-relevant structure. Any success under B1 therefore reflects robustness of the original proof to the refactoring itself rather than recovery capability. Operationally, B1 also determines the broken subset on which later recovery baselines are evaluated, so it should be interpreted primarily as the benchmark's post-refactoring viability check rather than as a recovery method in its own right.

B2: Syntactic transfer. B2 is a nearby-version continuity baseline. It transfers proof-relevant source artifacts from the seed program to the transformed version using rename-aware and diff-aware heuristics. In practice, this means that contracts, assertions, loop invariants, ghost code, and similar annotations are projected into the transformed file when a plausible surface correspondence can be established. The intent is not to perform semantic diagnosis, but to test how much refactoring-induced breakage can be removed by lightweight reuse of existing proof structure.

B3: LLM-only repair. B3 is the standardized LLM-only repair baseline. The model is fixed to `claude-sonnet-4-6` with temperature 0.2. Each instance is given up to five retries, verifier feedback is appended between retries, and success is counted only when the resulting program re-verifies in Dafny. This baseline is intentionally protocol-driven rather than open-ended: it measures what a strong lightweight generation strategy can recover under one fixed setup, not the maximum possible performance of language-model-based repair under unlimited prompt engineering. Appendix B records the prompt template, retry policy, and logging fields used to keep B3 comparable across benchmark cases.

4.3 Benchmark Axes and Analysis Units

The unit of analysis throughout this paper is a single Dataset A benchmark item: one verified seed program paired with one transformed nearby-version variant and its associated outcomes. Results are reported along four primary axes: seed-program stratum, refactoring class, baseline type, and post-refactoring verification or recovery outcome. This choice keeps the evaluation aligned with the benchmark specification introduced in Section 3. Optional toolchain or solver strata may be useful in future extensions, but they are not part of the core analysis here. The paper's primary question remains source-level proof brittleness under semantics-preserving refactoring, together with the amount of baseline recovery that can be achieved on those same benchmark items.

4.4 Execution Environment and Protocol

All experiments are recorded with a fixed execution environment that includes the Dafny version, underlying solver or toolchain version, hardware or container configuration, per-run timeout budget, and per-attempt logging fields for verifier wall-clock time and LLM token or cost accounting. These records are part of the benchmark metadata rather than an afterthought because some observed failures may depend on the exact verification environment in which they are replayed.

The protocol is intentionally simple and uniform. A transformed benchmark item is first generated and validated by the pipeline in Section 3. B1 is then run on the transformed program without edits. B2 is applied next to the subset that fails under B1 in order to test lightweight transfer of proof-relevant structure. When enabled, B3 is finally run on the remaining broken subset under the fixed prompt and retry protocol. After those runs complete, aggregate metrics are computed over the same underlying set of benchmark items, with `not_run` recorded explicitly whenever a baseline is intentionally deferred. This ordering ensures that all baselines are compared on a common evaluation substrate and that later analyses can attribute differences to recovery strategy rather than to changes in benchmark composition.

4.5 Metrics

This paper reports a compact set of metrics aimed at characterizing both brittleness and the size of the residual repair gap. At the benchmark level, it reports

composition statistics for Dataset A, including the number of seeds, transformed items, and items that survive to baseline evaluation. At the outcome level, it reports breakage rate by refactoring class and recovery rate for the baselines that are actually enabled in a given experiment tier. At the cost level, it records verifier wall-clock time together with B3 token usage and retry cost as lightweight effort proxies. At the analysis level, it reports the distribution of failure-mode labels and the residual repair gap that remains after the enabled recovery baselines have been applied. Where confidence intervals are eventually reported, they should be treated as an optional analysis layer attached to the final measurement pipeline rather than as a prerequisite for the paper's basic claims.

We use the following notation throughout the evaluation. Let N_{items} be the number of transformed instances retained by the benchmark pipeline and N_{eval} the number of instances that survive to baseline evaluation. For each refactoring class $class_i$, let $N_{attempted}(class_i)$ denote the number of attempted transformations and $N_{broken}(class_i)$ the number of those instances that fail verification immediately after transformation. Let N_{broken} denote the total broken subset induced by B1. For any recovery baseline $B_k \in \{B2, B3\}$, let $N_{recovered}(B_k \mid broken)$ denote the number of B1-broken instances repaired by that baseline. Let $\mathcal{B}_{enabled}$ denote the set of enabled recovery baselines in a given experiment tier and $N_{unrecovered}(\mathcal{B}_{enabled})$ the number of B1-broken instances left unresolved after those enabled baselines have been applied. Instances for which a baseline is intentionally deferred are recorded as `not_run` rather than folded into failure.

$$SR = \frac{N_{eval}}{N_{items}} \tag{1}$$

$$BR(class_i) = \frac{N_{broken}(class_i)}{N_{attempted}(class_i)} \tag{2}$$

$$RR(B_k \mid broken) = \frac{N_{recovered}(B_k \mid broken)}{N_{broken}}, \qquad B_k \in \{B2, B3\} \tag{3}$$

$$Gap_{residual}(\mathcal{B}_{enabled}) = \frac{N_{unrecovered}(\mathcal{B}_{enabled})}{N_{broken}} \tag{4}$$

To expose lightweight effort as well as success, we also report average verifier cost and average B3 token cost:

$$Cost_{ver}(B_k) = \frac{1}{N_{run}(B_k)} \sum_t time(t, B_k) \tag{5}$$

$$Cost_{tok}(B3) = \frac{1}{N_{run}(B3)} \sum_t tokens(t, B3) \tag{6}$$

These formulas make the interpretive structure of Section 5 explicit: the paper evaluates not only whether breakage occurs, but also how often enabled recovery baselines repair the B1-broken subset, how much of that subset remains unresolved, and at what cost. In early experiment tiers where B3 is deferred, the

paper therefore reports $RR(B2 \mid broken)$, records B3 as `not_run`, and interprets $Gap_{residual}$ relative to the enabled baseline set rather than to an idealized three-baseline configuration.

4.6 Empirical Success Criteria

P1 treats its empirical thesis as successful if two interpretation criteria are met. First, nearby-version proof breakage should be clearly non-trivial, operationalized here as at least three refactoring classes exhibiting breakage rate $\geq 20\%$. Second, the enabled recovery baselines should still leave a substantial residual repair gap, operationalized here as no enabled recovery baseline exceeding 70% recovery on the B1-broken subset. These criteria are not intended as universal thresholds for all future benchmarks. Instead, they give the paper a concrete way to judge whether semantics-preserving refactoring exposes a practically meaningful maintenance problem that is not already dissolved by simple replay, surface transfer, or lightweight generation.

5 Results

5.1 Benchmark Composition

We begin by summarizing the composition of Dataset A as an empirical artifact rather than only as input to the baseline comparison. Let N_{seed} denote the number of verified seed programs, N_{items} the number of transformed nearby-version instances retained after the generation pipeline, and N_{eval} the number of items that survive to baseline evaluation. The first summary table should report these quantities by seed-program stratum and by refactoring class, together with the ratio N_{eval}/N_{items}. This view establishes whether the benchmark has reasonable coverage across both source regimes and transformation types, and it also reveals whether some refactoring families are too sparse for stable table-level reporting in an early experiment tier. The broader pilot in Section 4 suggests that the composition table is informative once multiple strata are present, even before all seven classes are fully populated.

5.2 Breakage Rate by Refactoring Class

The core descriptive result is how often verification breaks after semantics-preserving refactoring. For each refactoring class $class_i$, we report the number of attempted transformations, the number of broken cases $N_{broken}(class_i)$, and the corresponding breakage rate $BR(class_i)$. If confidence intervals are ultimately computed, they should be attached to these rates rather than to isolated anecdotes. The key interpretive question is whether proof brittleness is concentrated in one pathological class or whether non-trivial breakage appears across multiple transformation families. Evidence for the latter would come from several classes exhibiting materially non-zero $BR(class_i)$ values and from those effects

Table 1. Planned benchmark-composition summary. The table is intended to show that Dataset A covers multiple seed strata and refactoring classes while retaining enough valid nearby-version pairs for baseline evaluation.

Stratum / class	Seeds	Items	Eval-ready
Tutorial / educational	N_{seed}^{tut}	N_{items}^{tut}	N_{eval}^{tut}
Benchmark / suite	N_{seed}^{bench}	N_{items}^{bench}	N_{eval}^{bench}
Proof-engineering-heavy	N_{seed}^{heavy}	N_{items}^{heavy}	N_{eval}^{heavy}
Rename	–	N_{items}^{rename}	N_{eval}^{rename}
Helper extraction	–	$N_{items}^{extract}$	$N_{eval}^{extract}$
Inlining	–	N_{items}^{inline}	N_{eval}^{inline}
Statement reordering	–	$N_{items}^{reorder}$	$N_{eval}^{reorder}$
Loop reshaping	–	N_{items}^{loop}	N_{eval}^{loop}
Contract factoring	–	$N_{items}^{contract}$	$N_{eval}^{contract}$
Ghost-code movement	–	N_{items}^{ghost}	N_{eval}^{ghost}
Total	N_{seed}	N_{items}	N_{eval}

persisting across more than one seed-program stratum rather than being confined to tutorial-scale examples. When some classes remain low frequency in an early pilot or first benchmark release, the paper should preserve them in the manifest-level data but may merge or defer them in the main text tables if that yields a clearer presentation.

5.3 Baseline Recovery Comparison

We next evaluate baseline recovery on the broken subset induced by B1. In practice, the informative recovery quantities in early experiments are $RR(B2 \mid broken)$ and, when enabled, $RR(B3 \mid broken)$. B1 is still reported, but primarily as the mechanism that defines which transformed instances require recovery rather than as a genuine repair baseline. The central comparison is therefore not only which enabled recovery baseline attains the highest conditional recovery rate, but also how their recovered subsets overlap, which baselines are intentionally deferred and thus recorded as `not_run`, and which benchmark items remain unrepaired after the enabled baseline set $\mathcal{B}_{enabled}$ has been applied. That remaining subset defines $Gap_{residual}(\mathcal{B}_{enabled})$. A strong version of the paper's empirical claim is that this residual gap remains substantial across multiple refactoring classes and seed strata even after nearby-version transfer and lightweight generation have been given a fair chance. If that pattern holds, then the benchmark shows not merely that breakage exists, but that simple recovery strategies leave meaningful nearby-version proof breakage unresolved.

5.4 Cost and Effort Proxies

The recovery comparison should be read together with lightweight cost proxies. For each enabled baseline, we record verifier wall-clock time, and for B3 we

Table 2. Planned breakage and recovery summary. The table is intended to show which refactoring classes are most brittle, how much of the B1-broken subset is recovered by enabled recovery baselines, and when a deferred baseline is explicitly recorded as `not_run` rather than conflated with failure. Low-frequency classes may be merged in the main text if early experiment tiers leave some rows too sparse for stable comparison.

Class	Att.	Brk.	$BR(class_i)$	$RR(B2 \mid broken)$	$RR(B3 \mid broken)$	
Rename	$N^{\text{rename}}_{attempted}$	$N^{\text{rename}}_{broken}$	$BR(\text{rename})$	$RR^{\text{rename}}(B2 \mid broken)$	$RR^{\text{rename}}(B3 \mid broken)$	ru
Helper extraction	$N^{\text{extract}}_{attempted}$	$N^{\text{extract}}_{broken}$	$BR(\text{extract})$	$RR^{\text{extract}}(B2 \mid broken)$	$RR^{\text{extract}}(B3 \mid broken)$	ru
Inlining	$N^{\text{inline}}_{attempted}$	$N^{\text{inline}}_{broken}$	$BR(\text{inline})$	$RR^{\text{inline}}(B2 \mid broken)$	$RR^{\text{inline}}(B3 \mid broken)$	ru
Statement reordering	$N^{\text{reorder}}_{attempted}$	$N^{\text{reorder}}_{broken}$	$BR(\text{reorder})$	$RR^{\text{reorder}}(B2 \mid broken)$	$RR^{\text{reorder}}(B3 \mid broken)$	ru
Loop reshaping	$N^{\text{loop}}_{attempted}$	N^{loop}_{broken}	$BR(\text{loop})$	$RR^{\text{loop}}(B2 \mid broken)$	$RR^{\text{loop}}(B3 \mid broken)$	ru
Contract factoring	$N^{\text{contract}}_{attempted}$	$N^{\text{contract}}_{broken}$	$BR(\text{contract})$	$RR^{\text{contract}}(B2 \mid broken)$	$RR^{\text{contract}}(B3 \mid broken)$	ru
Ghost-code movement	$N^{\text{ghost}}_{attempted}$	$N^{\text{ghost}}_{broken}$	$BR(\text{ghost})$	$RR^{\text{ghost}}(B2 \mid broken)$	$RR^{\text{ghost}}(B3 \mid broken)$	ru
Residual gap	N_{broken}	$N_{unrecovered}(\mathcal{B}_{enabled})$	–	$Gap_{residual}(\mathcal{B}_{enabled})$		

additionally record retry count, prompt-token usage, completion-token usage, and total token-derived cost. These quantities are not direct measures of human effort, but they do help distinguish cheap robustness from expensive recovery. In particular, the paper should report whether improvements in $RR(B3 \mid broken)$ come with large retry or token overhead relative to B1 and B2, and whether those costs vary meaningfully across refactoring classes or seed-program strata. If B3 is deferred in an early experiment tier, its result cells should appear explicitly as `not_run` rather than being left blank; this keeps the distinction between absence of execution and failed recovery visible in both tables and prose.

5.5 Failure-Mode Taxonomy and Residual Gap

To move beyond aggregate success rates, we analyze the broken subset through the failure-mode labels introduced in Section 3. Let L_{assert}, L_{inv}, L_{lemma}, $L_{trigger}$, L_{ghost}, and L_{tool} denote the counts associated with the six label categories. These correspond respectively to broken assertion flow, weakened loop invariants, brittle lemma or helper-application structure, trigger or quantifier sensitivity, ghost-code or proof-hint displacement, and unexplained or tool-sensitive failure. This view helps distinguish which kinds of nearby-version proof breakage are partially addressed by surface transfer and which persist into $Gap_{residual}(\mathcal{B}_{enabled})$ even after the enabled recovery baselines have been applied. In particular, a residual concentration in lemma-structure, invariant, or trigger-sensitive cases would motivate stronger symbolic repair in later work more convincingly than a residual dominated by simple renaming failures. If some labels appear only once or twice in an early experiment tier, they should remain visible in the raw results but may be summarized more coarsely in the main narrative to avoid overweighting pilot-scale noise.

At the same time, unrepaired cases should not automatically be interpreted as semantically deep failures. Some may still reflect solver brittleness, hidden

Table 3. Planned failure-mode distribution. The table is intended to show which proof-relevant failure categories dominate the broken subset and which categories continue to populate the residual repair gap after baseline recovery.

Failure-mode label	Broken subset	Residual gap subset
Broken assertion flow	L_{assert}	L^{res}_{assert}
Weakened loop invariants	L_{inv}	L^{res}_{inv}
Brittle lemma / helper structure	L_{lemma}	L^{res}_{lemma}
Trigger / quantifier sensitivity	$L_{trigger}$	$L^{\text{res}}_{trigger}$
Ghost-code / proof-hint displacement	L_{ghost}	L^{res}_{ghost}
Unexplained / tool-sensitive failure	L_{tool}	L^{res}_{tool}

context sensitivity, or benchmark-local effects that fall outside the expressive reach of B1–B3 without requiring fundamentally new semantics-aware reasoning. The taxonomy is therefore used here as an analysis device for interpreting the residual repair gap, not as a definitive theory of proof failure.

5.6 Takeaways

Taken together, the results should make one claim legible even before exact numbers are inserted: Dataset A exposes nearby-version proof breakage as a measurable maintenance problem, that problem appears across multiple semantics-preserving refactoring classes rather than in a single corner case, and the enabled recovery baselines still leave a non-trivial residual repair gap. In other words, the benchmark is designed to show not just that verified code can break under evolution, but that nearby-version proof maintenance remains a structured empirical problem after the easiest fixes have already been tried. That is the empirical substrate this paper aims to establish so that later repair work can be evaluated against it rather than inferred from isolated examples.

6 Threats to Validity

Seed-program representativeness. Dataset A may still over-represent tutorial-scale Dafny programs, particular annotation idioms, or proof styles that are easier to package into a controlled benchmark than large industrial developments. This matters primarily for generality rather than for internal causal interpretation: a benchmark can still cleanly measure brittleness within its source regimes while remaining incomplete as a census of all Dafny practice. The paper mitigates that risk by stratifying the seed corpus and by explicitly distinguishing educational examples from benchmark-style and proof-engineering-heavy programs, but this does not eliminate representativeness concerns. Any empirical pattern reported in the paper should therefore be read as evidence about the benchmarked source regimes rather than as a definitive statement about all verified Dafny code.

Refactoring realism. The benchmark uses controlled semantics-preserving refactorings instead of unconstrained maintenance history, and that choice inevitably reduces ecological realism. Real repository evolution mixes structural edits with partial rewrites, requirement changes, cleanup work, and toolchain drift in ways that are not captured by a controlled refactoring benchmark. This threat matters chiefly for external realism, not for the paper's central causal question. The paper makes this tradeoff deliberately: nearby-version pairs remain interpretable, reproducible, and suitable for clean breakage attribution, even though the benchmark does not attempt to cover the full messiness of real software evolution.

Semantic-preservation assumptions. Each transformation in Dataset A is intended to preserve behavior, but that intention does not rule out hidden preconditions, transformation bugs, or tool-specific corner cases. A refactoring that is semantics preserving in an informal sense may still interact with Dafny parsing, name resolution, or proof-relevant structure in unexpected ways. This threat bears directly on causal interpretation because it can make an observed failure look like proof brittleness when it is partly a benchmark-construction artifact. The paper reduces this threat through seed validation, transformation preconditions, post-transformation verification replay, and exclusion of malformed cases. Even so, some residual risk remains that a subset of observed failures reflect benchmark-construction artifacts rather than pure proof brittleness.

Baseline sensitivity. The recovery outcomes for B2 and B3 depend on fixed implementation choices. For B2, rename heuristics, diff granularity, and transfer policy influence how much nearby-version continuity is recovered. For B3, prompt wording, retry limits, timeout budgets, and verifier-feedback handling influence the observed recovery rate. This is an interpretation threat rather than a benchmark-definition threat: the paper can still establish a residual repair gap, but that gap is conditional on one fixed baseline protocol. The paper addresses this by fixing a single protocol and reporting results under that protocol rather than informally tuning per instance. Even so, the measured residual repair gap should be understood as the gap that remains after these particular baseline instantiations, not as an absolute upper bound on all possible lightweight repair strategies.

Verifier and toolchain sensitivity. Some broken or unrecovered cases may reflect sensitivity of the verifier or underlying solver rather than purely source-level proof-structural difficulty. The same transformed program may behave differently under another Dafny release, solver version, or execution environment. This threat weakens causal precision for some residual failures, especially those classified as unexplained or tool-sensitive. The paper records the toolchain and environment explicitly, but it does not attempt to isolate tool-version instability as a primary benchmark axis. Accordingly, it can establish that nearby-version proof breakage is present under a fixed and documented environment, while leaving a fuller account of environment-sensitive variation to later benchmark extensions.

External validity. The conclusions of this paper should not be generalized too aggressively. Dafny is an auto-active verifier with source-visible proof structure, and the maintenance phenomena observed here may not transfer directly to interactive theorem provers, other SMT-backed verifiers, or verification ecosystems with different proof abstractions. Likewise, controlled semantics-preserving refactorings are only one slice of software evolution, and B1–B3 cover only a narrow part of the overall repair design space. The main claim is therefore intentionally modest: Dataset A provides a reproducible maintenance-centred benchmark that makes nearby-version proof breakage measurable and exposes a residual repair gap under simple baselines.

7 Related Work

Refactoring and software evolution. Classical refactoring work established both the breadth of refactoring activities and their importance for long-term software maintenance [14]. Later empirical studies showed that refactoring decisions arise regularly in open-source projects and span a wide range of structural transformations rather than a handful of stylized edits [10, 18]. More generally, software-engineering research has emphasized that software artifacts coevolve over time, and that understanding maintenance behavior often requires repository- or revision-oriented analysis rather than one-shot task evaluation [25]. P1 builds directly on this maintenance lens, but shifts the target artifact from code-and-tests alone to verified programs whose code, contracts, and proof hints evolve together.

Verification engineering and proof maintenance. Work on proof maintenance has mostly appeared in theorem-proving settings. Whiteside et al. formalize proof-script refactoring as a semantics-preserving restructuring activity [20], and Whiteside's later thesis expands this line into a larger framework for refactoring proofs [21]. Tool-oriented work such as Tactician shows that proof refactoring can also be operationalized in interactive settings [1]. In the Dafny ecosystem, adjacent work has instead focused on proof guidance and verification support: reusable verification patterns [7], testing and differential validation of the Dafny toolchain [9, 2], and empirical studies of how auto-active verification affects development practice [16]. These papers make two points clear: proof-oriented artifacts are structured enough to merit systematic support, and maintainability matters in practice. However, they do not provide a benchmark for refactoring-induced proof brittleness in an SMT-backed verifier like Dafny.

Automated repair and learning-based assistance. Automated software repair supplies the closest high-level analogy for this paper's baseline design, especially generate-and-validate workflows and search over candidate fixes [6, 22]. On the proof side, systems such as TacTok and Passport show that proof corpora can support meaningful automation and synthesis [4, 17]. Meanwhile, pretrained and generative code models such as CodeBERT, GraphCodeBERT, and InCoder

have become standard reference points for learning-based code generation and repair [3, 8, 5]. Recent Dafny-specific work extends this trend toward verified code generation and benchmarking, including LLM-assisted synthesis of verified Dafny methods [15], DafnyBench [13], DafnyComp [23], and Re:Form's RL-oriented formal verification pipeline [24]. This paper uses that literature primarily as baseline and context: it does not propose a repair engine, but rather quantifies how much of the maintenance problem remains after simple baselines such as re-verification, syntactic transfer, and LLM-only repair.

Gap and positioning of P1. Dafny has long been used in benchmark-style verification studies [11, 12], and recent work shows growing interest in Dafny-oriented benchmarks for synthesis, compositional reasoning, testing, and benchmark generation [13, 19, 23]. Likewise, prior work has studied refactoring broadly, proof refactoring in interactive provers, and repair or synthesis through both symbolic and learned techniques. What is still missing is a *maintenance-centred* benchmark and empirical study centered on *semantics-preserving refactoring of already verified Dafny programs*, together with baseline evidence on how much nearby-version proof breakage simple recovery strategies can repair. This paper targets that missing slice of the space and, by doing so, provides a reusable evaluation substrate for later repair work without collapsing benchmark construction into repair-system design.

8 Conclusion

This paper argues that proof brittleness under semantics-preserving refactoring is a measurable maintenance problem in Dafny and that this problem deserves its own benchmark-and-evaluation substrate. The main contribution is therefore not a repair algorithm, but a maintenance-centred benchmark specification for nearby-version proof breakage, together with an empirical evaluation that measures how often verification fails under controlled refactorings and how much of that failure can be recovered by simple baselines.

The intended empirical outcome is a clear residual repair gap: even after re-verification, syntactic transfer, and LLM-only repair, a substantial class of nearby-version failures should remain unresolved. By organizing those failures by refactoring class, source stratum, and proof-oriented failure mode, the paper turns an often anecdotal maintenance complaint into a reproducible benchmark problem. That is the paper's standalone value: it makes proof maintenance under evolution measurable rather than merely discussable.

Bridge to later work. The benchmark, taxonomy, and baseline results produced here also make later symbolic repair work measurable rather than merely imaginable. In that sense, this paper is both a complete empirical study and the evaluation substrate on which later work can test whether structured repair genuinely closes the residual gap identified here.

A Artifact and Reproducibility Details

Artifact contents. The artifact package includes the benchmark generator **[MISSING: generator package]**, replay harness **[MISSING: replay entrypoint]**, baseline implementations **[MISSING: baseline packages]**, raw logs **[MISSING: log bundle]**, and analysis scripts **[MISSING: analysis entrypoint]**. The release package URL is **[MISSING: Artifact URL]**.

Environment. The pinned Dafny version is **[MISSING: Dafny version]**, the solver version is **[MISSING: Solver version]**, the operating system or container base image is **[MISSING: Base image]**, and the hardware assumptions are **[MISSING: Hardware assumptions]**. The Docker image tag used for replay is **[MISSING: Docker tag]**.

Replay workflow. The artifact replay workflow is organized around the following commands:

1. build Dataset A with `[MISSING: build command]`,
2. run B1, B2, and B3 with `[MISSING: replay command]`,
3. regenerate tables and figures with `[MISSING: analysis command]`,
4. package the final results with `[MISSING: package command]`.

Anonymization and release. For double-blind submission, the following elements must be removed or anonymized: **[MISSING: repo identifiers]**, **[MISSING: organization names]**, and **[MISSING: hosting metadata]**. The public artifact released after acceptance will additionally contain **[MISSING: public release contents]**.

B Standardized LLM Prompt and Retry Protocol

Prompt template. The exact B3 prompt should remain fixed across benchmark items except for the per-instance fields listed below. The working draft template is:

[MISSING: Exact B3 prompt]

The per-instance fields that vary across cases are:

- old program: **[MISSING: Old-program field]**,
- transformed program: **[MISSING: Transformed-program field]**,
- verifier error: **[MISSING: Verifier-error field]**,
- repair instruction: **[MISSING: Repair instruction]**.

Retry policy. The fixed retry protocol for B3 is:

- maximum of 5 retries,
- verifier feedback appended between retries,
- temperature fixed at 0.2,
- success only on proof-checked output.

The exact verifier-feedback wrapper appended between retries is **[MISSING: Verifier-feedback wrapper]**.

Logging fields. Each attempt records the following fields:

- model identifier: **[MISSING: Model ID]**,
- prompt tokens,
- completion tokens,
- wall-clock time,
- verifier outcome,
- retry count.

Protocol stability. To preserve comparability across benchmark cases, the following elements must remain fixed: **[MISSING: Prompt prefix]**, **[MISSING: Execution environment]**, and **[MISSING: Verifier invocation]**.

References

1. Adams, M.: Refactoring proofs with tactician. In: SEFM 2015 Collocated Workshops (2015), https://proof-technologies.com/papers/tactician_hofm2015.html
2. Fedchin, A., Dean, T., Foster, J.S., Mercer, E., Rakamaric, Z., Reger, G., Rungta, N., Salkeld, R., Wagner, L., Waldrip, C.: A toolkit for automated testing of dafny. In: NASA Formal Methods (2023), https://www.amazon.science/publications/a-toolkit-for-automated-testing-of-dafny
3. Feng, Z., et al.: Codebert: A pre-trained model for programming and natural languages. EMNLP Findings (2020), https://arxiv.org/abs/2002.08155
4. First, E., Brun, Y., Guha, A.: Tactok: semantics-aware proof synthesis. Proceedings of the ACM on Programming Languages (2020). https://doi.org/10.1145/3428299, https://doi.org/10.1145/3428299
5. Fried, D., et al.: Incoder: A generative model for code infilling and synthesis. ICLR (2022), https://arxiv.org/abs/2204.05999
6. Gazzola, L., Micucci, D., Mariani, L.: Automatic software repair: A survey. IEEE Transactions on Software Engineering (2017). https://doi.org/10.1109/TSE.2017.2755013, https://doi.org/10.1109/TSE.2017.2755013
7. Grov, G., Lin, Y., Tumas, V.: Mechanised verification patterns for dafny. In: FM 2016: Formal Methods. Lecture Notes in Computer Science, vol. 9995 (2016). https://doi.org/10.1007/978-3-319-48989-6_20, https://doi.org/10.1007/978-3-319-48989-6_20
8. Guo, D., et al.: Graphcodebert: Pre-training code representations with data flow. ICLR (2020), https://arxiv.org/abs/2009.08366
9. Irfan, A., Porncharoenwase, S., Rakamaric, Z., Rungta, N., Torlak, E.: Testing dafny (experience paper). In: ISSTA (2022), https://ahmed-irfan.github.io/papers/issta22.pdf
10. Kim, M., et al.: Refactoring in the wild: A study of refactoring decisions in open source projects. In: ICSE (2012). https://doi.org/10.1109/ICSE.2012.6227218, https://doi.org/10.1109/ICSE.2012.6227218
11. Leino, K.R.M.: Dafny: An automatic program verifier for functional correctness. In: LPAR-16 (2010). https://doi.org/10.1007/978-3-642-17511-4_20, https://doi.org/10.1007/978-3-642-17511-4_20

12. Leino, K.R.M.: Dafny meets the verification benchmarks challenge. In: VSTTE (2010), https://www.microsoft.com/en-us/research/publication/dafny-meets-the-verification-benchmarks-challenge/
13. Loughridge, C., Sun, Q., Ahrenbach, S., Cassano, F., Sun, C., Sheng, Y., Mudide, A., Misu, M.R.H., Amin, N., Tegmark, M.: Dafnybench: A benchmark for formal software verification. arXiv preprint arXiv:2406.08467 (2024), https://arxiv.org/abs/2406.08467
14. Mens, T., Tourwe, T.: A survey of software refactoring. IEEE Transactions on Software Engineering (2004). https://doi.org/10.1109/TSE.2004.1265817, https://doi.org/10.1109/TSE.2004.1265817
15. Misu, M.R.H., Lopes, C.V., Ma, I., Noble, J.: Towards ai-assisted synthesis of verified dafny methods. arXiv preprint arXiv:2402.00247 (2024), https://arxiv.org/abs/2402.00247
16. Mugnier, E., Zhou, Y., Jhala, R., Coblenz, M.: On the impact of formal verification on software development. Proceedings of the ACM on Programming Languages **9**(OOPSLA2) (2025). https://doi.org/10.1145/3763181, https://doi.org/10.1145/3763181
17. Sanchez-Stern, A., First, E., Zhou, T., Kaufman, Z., Brun, Y., Ringer, T.: Passport: Improving automated formal verification using identifiers. ACM Transactions on Programming Languages and Systems (2023). https://doi.org/10.1145/3593374, https://doi.org/10.1145/3593374
18. Tsantalis, N., et al.: A large-scale study on the usage of refactoring operations. IEEE Transactions on Software Engineering (2014). https://doi.org/10.1109/TSE.2014.2301518, https://doi.org/10.1109/TSE.2014.2301518
19. Wang, C., Scazzariello, M., Kostic, D., Chiesa, M.: Toward automated, contamination-free dafny benchmark generation. Dafny 2026 workshop preprint (2026), https://popl26.sigplan.org/details/dafny-2026-papers/17/Toward-Automated-Contamination-free-Dafny-Benchmark-Generation
20. Whiteside, I., Aspinall, D., Dixon, L., Grov, G.: Towards formal proof script refactoring. In: Intelligent Computer Mathematics. Lecture Notes in Computer Science, vol. 6824 (2011). https://doi.org/10.1007/978-3-642-22673-1_18, https://doi.org/10.1007/978-3-642-22673-1_18
21. Whiteside, I.J.: Refactoring Proofs. Ph.D. thesis, The University of Edinburgh (2013), https://era.ed.ac.uk/handle/1842/7970
22. Xia, C.S., et al.: Automated program repair with pretrained language models. ICSE (2023). https://doi.org/10.1109/ICSE48619.2023.00072, https://doi.org/10.1109/ICSE48619.2023.00072
23. Xu, X., Li, X., Qu, X., Fu, J., Yuan, B.: Local success does not compose: Benchmarking large language models for compositional formal verification. ICLR (2026), https://openreview.net/forum?id=y4kAMUBqLq
24. Yan, C., Che, F., Huang, X., Xu, X., Li, X., Li, Y., Qu, X., Shi, J., Lin, C., Yang, Y., Yuan, B., Zhao, H., Qiao, Y., Zhou, B., Fu, J.: Re:form – reducing human priors in scalable formal software verification with rl in llms: A preliminary study on dafny. OpenReview preprint (2025), https://openreview.net/forum?id=cAQmIS4GOe
25. Zaidman, A., van Rompaey, B., Demeyer, S., et al.: Studying the co-evolution of production and test code in open source and industrial developer test processes through repository mining. Empirical Software Engineering (2010). https://doi.org/10.1007/s10664-010-9143-7, https://doi.org/10.1007/s10664-010-9143-7